

High-Level Design Report

1. Introduction

Recent advancements in deep learning and computer vision have enabled highly capable image editing techniques such as semantic segmentation, object removal, style transfer, and automated enhancement. Despite these technical advances, a significant gap remains between what modern AI models can achieve and what end users can practically utilize. Existing professional tools require extensive technical knowledge and manual effort, while fully automated AI-based applications often lack user control and correction mechanisms.

From a system engineering perspective, this gap represents a usability and control problem rather than a purely algorithmic limitation. Many AI-powered image editors treat model outputs as final results, even though real-world images frequently contain edge cases where automated predictions fail—such as complex object boundaries, hair regions, or overlapping foreground elements. Users are therefore forced to either accept imperfect results or switch to professional tools, undermining the benefits of automation.

Invisio is proposed as a web-based image editing system that addresses this problem through a **hybrid AI-assisted design approach**. Instead of positioning artificial intelligence as an infallible black box, the system combines automated AI processing with limited, structured user intervention. This human-in-the-loop strategy allows users to correct minor inaccuracies at the mask level while preserving the simplicity and performance advantages of AI-driven editing.

Beyond functionality, Invisio is designed with **privacy, data minimization, and ethical responsibility** as core architectural principles. User-uploaded images are processed only within a temporary session context and are never stored persistently. This design directly addresses privacy risks commonly associated with cloud-based image editing platforms and aligns with modern data protection regulations such as GDPR and KVKK.

1.1. Purpose of the system

The purpose of the Invisio system is to provide a **web-based, AI-assisted image editing environment** that enables non-technical users to perform complex visual editing tasks in an efficient, accurate, and responsible manner. The system is designed to bridge the gap between professional-grade image editing capabilities and the usability expectations of everyday users by combining automation with controlled user interaction.

Existing image editing solutions generally fall into one of two extremes. Professional tools offer fine-grained control but require significant technical expertise, long learning periods, and time-consuming workflows. In contrast, fully automated AI-based applications prioritize simplicity and speed but provide little to no control when AI-generated results are incorrect or visually suboptimal. These limitations often force users to accept imperfect outputs or abandon automated tools altogether.

Invisio is intentionally positioned between these extremes through the adoption of a **hybrid editing model**. Rather than treating artificial intelligence as a fully autonomous decision-maker, the system uses AI as the primary processing engine while allowing users to intervene only when necessary. Artificial intelligence performs tasks such as segmentation, enhancement, inpainting, and style transfer, whereas user interaction is limited to constrained refinement actions. This design ensures that user input remains **corrective rather than technical or creative**, preserving both usability and system performance.

From a system design perspective, Invisio is intended to operate as a **stateless, session-oriented processing platform**. User-uploaded images and all intermediate processing results exist only for the duration of an active editing session and are discarded immediately after processing is completed. This approach directly supports the system's purpose by minimizing data exposure, reducing security risks, and simplifying compliance with data protection regulations such as GDPR and KVKK.

Another fundamental purpose of the system is to **abstract the complexity of AI models and image processing pipelines** away from the user. Advanced computational processes, model selection, and parameter tuning are encapsulated entirely within the backend architecture and are not exposed through the user interface. Users interact with the system exclusively through high-level actions that reflect editing intent rather than technical implementation details, allowing effective use without specialized knowledge.

In summary, the purpose of the Invisio system can be characterized by the following objectives:

- to enable fast, accessible, and AI-based image editing for non-expert users,
- to balance automation with limited user control through a hybrid human-in-the-loop approach,
- to ensure privacy and ethical data handling through session-based, non-persistent processing,
- and to provide a scalable architectural foundation suitable for modern web environments.

1.2. Design goals

The design of the Invisio system is guided by a set of high-level design goals that define the architectural direction of the system and shape key design decisions. These goals are derived from the analysis of existing image editing solutions, the characteristics of the target user group, and the technical and ethical constraints associated with AI-based image processing systems.

Rather than describing individual features, the following goals establish the principles that govern system behavior, interaction style, data handling, and long-term evolution.

- **Hybrid AI–Human Editing Model:** Invisio is designed to balance automation with limited user control by adopting a hybrid AI–human editing approach. Artificial intelligence performs the primary image processing operations, while users are allowed to intervene only when necessary through constrained refinement mechanisms. This design goal avoids fully opaque automation and improves result accuracy without introducing professional-level editing complexity.
- **Abstraction of Technical Complexity:** The system abstracts the internal complexity of AI models, processing pipelines, and parameter configurations from end users. Advanced operations such as segmentation, inpainting, enhancement, and style transfer are exposed exclusively as high-level editing actions. This goal ensures usability for non-technical users and prevents errors caused by low-level configuration.
- **Privacy-by-Design and Data Minimization:** User privacy is treated as a core design constraint. Invisio processes all images within a temporary session context and avoids persistent storage of user data by default. By minimizing data retention and limiting data lifetime, the system reduces legal, ethical, and security risks and supports compliance with data protection regulations such as GDPR and KVKK.
- **Stateless Interaction Model:** The system follows a stateless interaction model in which each image processing request is handled independently of previous interactions. No long-term user state or image data is retained across sessions. This design goal improves scalability, simplifies fault isolation, and aligns with the system’s privacy-oriented architecture.
- **Performance-Aware Web-Based Processing:** Given the computational demands of AI-based image editing, the system is designed to provide predictable and bounded response times in a web environment. User interactions are structured around discrete processing steps rather than continuous real-time manipulation, ensuring responsiveness while efficiently utilizing backend computational resources.
- **Separation of Responsibilities:** The system architecture clearly separates user interaction logic, request orchestration, and AI processing responsibilities. This separation improves maintainability, testability, and clarity of the overall system design, and prevents tight coupling between interface components and model execution logic.
- **Model-Agnostic and Extensible Design:** Invisio is designed to remain independent of specific AI models or frameworks. Image processing capabilities are encapsulated in a way that allows models to be replaced, upgraded, or extended without requiring fundamental architectural changes. This goal supports future evolution and experimentation as AI techniques advance.
- **Asynchronous and Non-Blocking User Experience:** AI-based processing tasks are executed asynchronously to prevent blocking the user interface. The system is designed to maintain responsiveness and provide clear feedback during long-running operations, improving perceived performance and overall user experience.
- **Fault Tolerance and Graceful Degradation:** The system is designed to handle AI processing failures, invalid inputs, or resource limitations in a controlled manner. Instead of failing silently or crashing, Invisio provides clear error handling and recovery behavior, ensuring reliability under non-ideal conditions.
- **Ethical and Responsible AI Usage:** Invisio incorporates ethical considerations directly into system design. AI-generated edits are intended to be transparent and controlled,

avoiding misleading or harmful outcomes. This goal ensures responsible use of image manipulation technologies and aligns the system with contemporary ethical AI principles.

- **Security-by-Default:** Security mechanisms such as secure communication, controlled access to processing resources, and minimal data exposure are integrated into the system by default. This goal ensures that security is an inherent property of the system rather than an optional or external addition.

1.3. Definitions, acronyms, and abbreviations

This section defines key terms, acronyms, and abbreviations used throughout the Invisio project documentation. Establishing a shared and precise terminology ensures clarity in system design discussions and prevents ambiguity in architectural and functional descriptions.

- **Artificial Intelligence (AI):**
Computational methods that enable machines to perform tasks typically requiring human intelligence, such as visual understanding, pattern recognition, and automated decision-making.
- **Machine Learning (ML):**
A subfield of artificial intelligence focused on enabling systems to learn patterns and relationships from data without being explicitly programmed.
- **Deep Learning:**
A class of machine learning techniques based on multi-layer neural network architectures, widely used for image analysis, segmentation, and generative modeling.
- **Computer Vision:**
A domain of artificial intelligence concerned with enabling machines to interpret and analyze visual information from digital images.
- **Convolutional Neural Network (CNN):**
A type of deep neural network architecture commonly used for image processing tasks such as feature extraction, classification, and segmentation.
- **Generative Adversarial Network (GAN):**
A generative model composed of two competing neural networks—a generator and a discriminator—used to synthesize realistic images.
- **Inference:**
The execution phase of a trained machine learning model in which predictions or transformations are produced from input data.
- **Image Segmentation:**
The process of dividing an image into meaningful regions, typically at the pixel level, to support tasks such as object removal or background separation.
- **Mask:**
A binary or multi-valued region map that defines which areas of an image are subject to modification or preservation during processing.

- **Inpainting:**
An image processing technique used to reconstruct removed or missing regions of an image based on surrounding visual context.
- **Neural Style Transfer:**
A technique that applies the visual style of one image to the content structure of another using deep neural networks.
- **Human-in-the-Loop:**
A system design approach in which limited human input is incorporated into an automated process to improve reliability or accuracy.
- **Hybrid Editing Workflow:**
An image editing approach that combines automated AI-based processing with optional user-driven refinement steps.
- **Session-Based Processing:**
A data handling model in which user data exists only for the duration of an active interaction session and is deleted immediately afterward.
- **Stateless System:**
A system architecture in which each request is handled independently, without reliance on previously stored user-specific state.
- **Volatile Memory:**
Temporary memory used for processing data that is cleared once the execution context ends, ensuring no long-term data retention.
- **Privacy-by-Design:**
A system design principle that integrates data protection and privacy considerations directly into architectural decisions rather than adding them as external controls.
- **Transport Layer Security (TLS):**
A cryptographic protocol used to secure data transmission between client and server.
- **GDPR (General Data Protection Regulation):**
A European Union regulation governing the lawful processing and protection of personal data.
- **KVKK (Kişisel Verilerin Korunması Kanunu):**
Türkiye's data protection law regulating the collection, processing, and security of personal data.

1.4. Overview

This section provides a high-level overview of the Invisio system by summarizing its overall structure, interaction flow, and design philosophy. The objective is to present how the system operates as a whole, without delving into implementation details, while maintaining consistency with the system purpose and design goals defined in previous sections.

Invisio is designed as a **web-based, AI-assisted image editing system** that follows a client–server interaction model. All user interactions take place through a browser-based interface, while computationally intensive image processing tasks are executed on the server side. This separation enables consistent performance across heterogeneous user devices and centralizes control over AI processing resources.

From the user's perspective, the system operates through a **task-oriented workflow**. Users upload an image, select one or more AI-based editing operations, optionally refine AI-generated results when necessary, preview the output, and download the final image. At no point are users exposed to low-level image processing controls or model-specific parameters. This interaction style directly reflects the system's purpose of abstracting technical complexity while preserving meaningful control.

Internally, the system follows a **stateless, session-oriented processing model**. Each image editing request is handled within a temporary session context, during which the image and all intermediate results are processed in volatile memory. Once processing is completed and the result is delivered to the user, all session data is discarded. This approach supports privacy-by-design principles and minimizes risks associated with persistent data storage.

The system architecture is organized to support **clear separation of responsibilities**. User interface components are responsible solely for interaction and visualization, while backend components manage request coordination and AI processing workflows. Image transformation logic is encapsulated within dedicated processing modules, allowing individual AI models to be updated or replaced without affecting the overall system structure.

To ensure usability and responsiveness in a web environment, AI-based operations are executed **asynchronously**. Users receive immediate feedback regarding processing state, and long-running tasks do not block the user interface. Error conditions such as invalid input, processing failure, or resource limitations are handled gracefully, ensuring predictable system behavior under non-ideal conditions.

In summary, Invisio is designed as a modular, privacy-aware, and extensible AI-powered image editing platform. Its high-level structure reflects the design goals established earlier by combining automated AI processing with limited user intervention, enforcing strict data minimization, and maintaining a clear architectural separation between interaction, coordination, and processing responsibilities.

2. Current software architecture (if any)

The Invisio project is built on top of an existing basic software structure that was created during early experimentation and model evaluation phases. At this stage, the system includes pre-trained GAN-based image processing models and pre-trained detection models for face enhancement (which will be fine-tuned by our team with specific face datasets), a simple user interface, and API endpoints that allow these models to be accessed and tested.

Currently, the image processing functionality is provided by individual deep learning models, mainly based on Generative Adversarial Networks (GANs) and similar convolutional architectures. Each model is designed to perform a specific task such as image enhancement, object removal, or style transfer. These models are exposed through basic API calls, which makes it possible to send an image, run inference, and receive the output.

The existing user interface is minimal and mainly serves as a demonstration and testing environment. It allows users to upload images and apply selected AI-based operations.

From a software architecture point of view, the current system can be described as a collection of loosely connected components rather than a fully integrated platform. While this structure is sufficient for testing individual models and validating their outputs, it is not suitable for a scalable, maintainable, or privacy-focused system.

For this reason, the existing setup is considered a starting point rather than a final solution. The proposed high-level architecture aims to reorganize and extend these components into a more systematic and modular structure, allowing the system to support more complex workflows, better user experience, and stronger privacy guarantees.

3. Proposed software architecture

The proposed software architecture of the Invisio system is designed with a strong emphasis on modularity, separation of concerns, and responsibility isolation. Given the complexity of AI-based image processing workflows and the ethical requirements associated with handling user-generated images, the architecture aims to provide a clear structural organization that supports scalability, maintainability, and controlled system evolution.

Rather than coupling user interaction logic directly with AI model execution, the system adopts a layered architectural model. Each layer encapsulates a specific responsibility and interacts with other layers through well-defined interfaces. This design choice reduces interdependencies between components and allows individual layers to evolve independently as system requirements change.

The architecture is conceptually divided into the following layers:

- **Presentation Layer**

The Presentation Layer is responsible for all user-facing interactions and visual feedback. This includes image upload mechanisms, feature selection interfaces, preview rendering, progress indicators, and result download functionality. The layer is designed to remain lightweight and device-independent, ensuring that users can access the system through standard web browsers without requiring specialized hardware or software installations. Importantly, this layer does not contain any AI logic or image processing responsibilities, which prevents performance bottlenecks and simplifies interface maintenance.

- **Control Layer**

The Control Layer acts as the central coordination mechanism of the system. It manages the flow of requests between the Presentation Layer and backend processing components, enforces validation rules, and determines which processing paths should be executed for a given user action. By centralizing decision-making logic, this layer ensures consistent system behavior, simplifies error handling, and prevents duplication of control logic across subsystems.

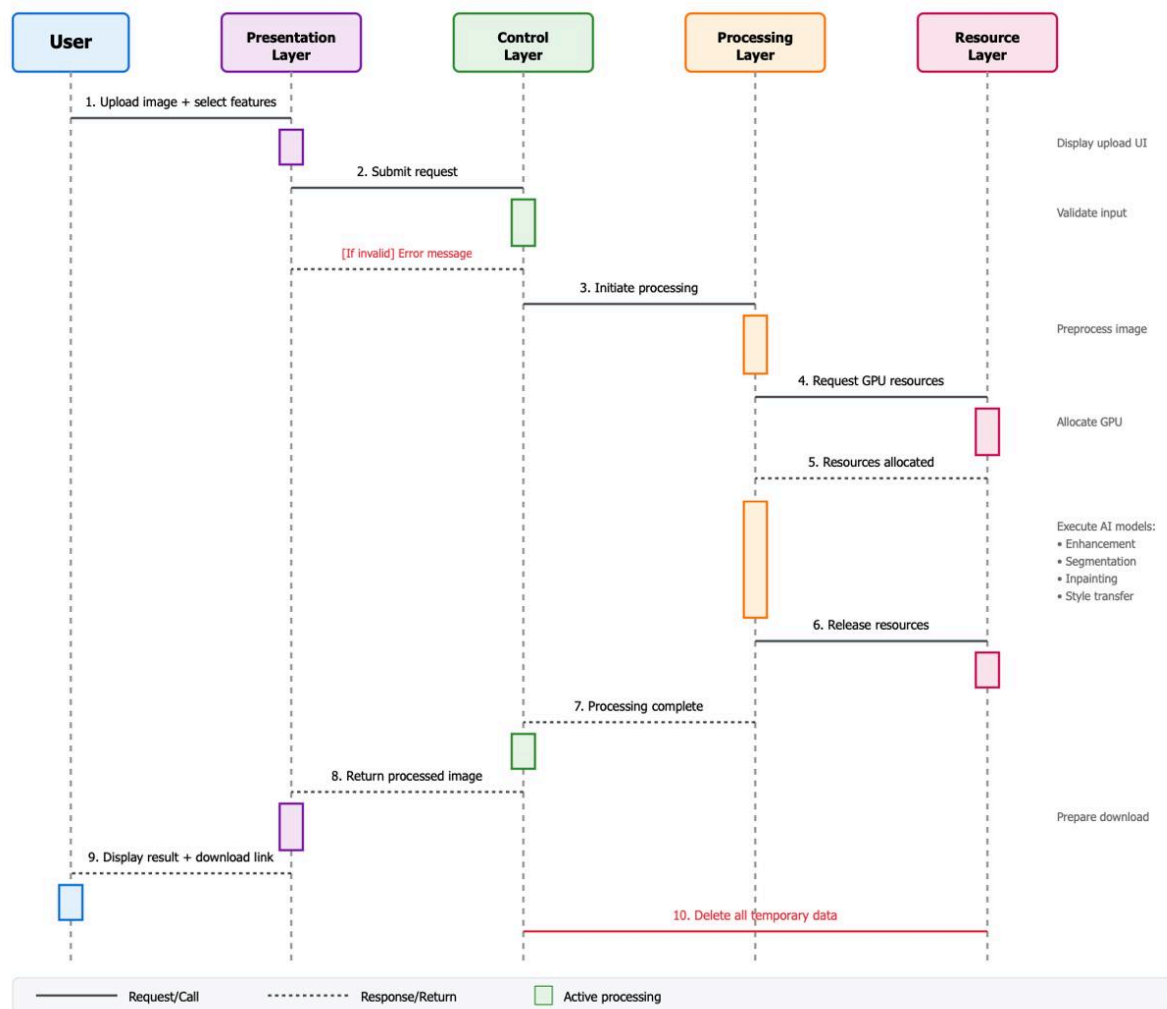
- **Processing Layer**

The Processing Layer encapsulates all AI-driven image transformation logic. This includes preprocessing steps, invocation of AI models, and postprocessing of results. Each image editing operation—such as object removal, enhancement, or style transfer—is treated as a discrete processing task within this layer. Encapsulating these operations allows the system to maintain a clean abstraction between high-level user intent and low-level computational execution.

- **Resource Layer**

The Resource Layer abstracts access to computational infrastructure, including CPU and GPU resources. It is responsible for managing execution environments, allocating computational capacity, and ensuring that AI workloads are executed efficiently. By isolating hardware concerns within this layer, the architecture remains flexible and portable across different deployment environments.

Overall, this layered model supports architectural clarity, improves testability, and provides a strong foundation for future extensions of the system.



3.1. Overview

This section provides a high-level overview of how the logical architecture of the Invisio system maps onto the underlying execution environment. The purpose of this overview is not to describe implementation details, but to illustrate how architectural responsibilities are distributed across client-side and server-side components in a coherent and scalable manner.

Invisio follows a distributed client–server architecture in which user interaction and computational processing are intentionally separated. All user-facing activities, such as image selection and result visualization, are handled on the client side through a web-based interface. In contrast, computationally intensive tasks related to AI-based image processing are executed exclusively on server-side infrastructure.

This separation serves multiple purposes. First, it enables consistent performance regardless of the user’s device capabilities, as AI processing is centralized on controlled server environments. Second, it simplifies scalability by allowing computational resources to be scaled independently of the user interface. Third, it supports the system’s privacy-oriented design by ensuring that sensitive image data is processed only within trusted server environments.

The client–server separation also enables efficient utilization of specialized hardware, such as GPUs, which are typically unavailable on end-user devices. By centralizing AI processing, the system avoids reliance on heterogeneous client-side computational capabilities and ensures predictable performance across a wide range of usage scenarios.

3.2. Subsystem decomposition

From an analytical standpoint, the Invisio system is decomposed into subsystems based on functional responsibility rather than technical implementation details. This decomposition reflects the system’s conceptual organization and facilitates clear reasoning about system behavior.

The primary subsystems are defined as follows:

- **User Interaction Subsystem**
Responsible for managing all user-facing interactions, including image upload, feature selection, preview display, and result download. This subsystem focuses exclusively on usability and interaction flow.
- **Request Management Subsystem**
Handles incoming requests, performs validation checks, and orchestrates the execution of processing tasks. It ensures that requests are routed correctly and that invalid inputs are rejected early.
- **Image Processing Subsystem**
Manages preprocessing and postprocessing operations required for image editing tasks. This includes format normalization, resizing, and result preparation.

- **Model Execution Subsystem**
Executes AI models responsible for image enhancement, segmentation, inpainting, and style transfer. This subsystem encapsulates model invocation and inference logic.
- **Output Generation Subsystem**
Prepares processed images for visualization and download. It ensures that results are delivered in a user-friendly and consistent format.
- **Monitoring and Error Handling Subsystem**
Tracks system state, detects failures, and provides meaningful feedback to users. This subsystem improves reliability and transparency.

This functional decomposition enables parallel development, limits fault propagation, and supports long-term system maintainability.

3.3. Hardware/software mapping

The system is designed to operate in a distributed execution environment with a clear separation between client-side interaction components and server-side computational services.

Client-side components are executed within a standard web browser and are responsible solely for user interaction tasks, such as image upload, feature selection, and result visualization. No specialized software installations or hardware accelerators are required on the user's device, ensuring broad accessibility across heterogeneous platforms.

Server-side components are deployed on infrastructure equipped with high-performance computing resources, particularly GPU-enabled environments. AI-based image processing tasks are exposed through server-side processing services that receive image processing requests from the client layer and execute model inference in a controlled execution context. This approach allows computational workloads to be centralized and managed independently of the user interface.

By routing AI processing requests to dedicated GPU-backed services, the system ensures consistent performance and efficient utilization of specialized hardware. The client layer remains agnostic to the underlying execution details, interacting only through well-defined service interfaces. This separation allows backend processing services to scale independently, supports flexible deployment across different operating systems and cloud providers, and enables future architectural evolution without impacting client-side components.

3.4. Persistent data management

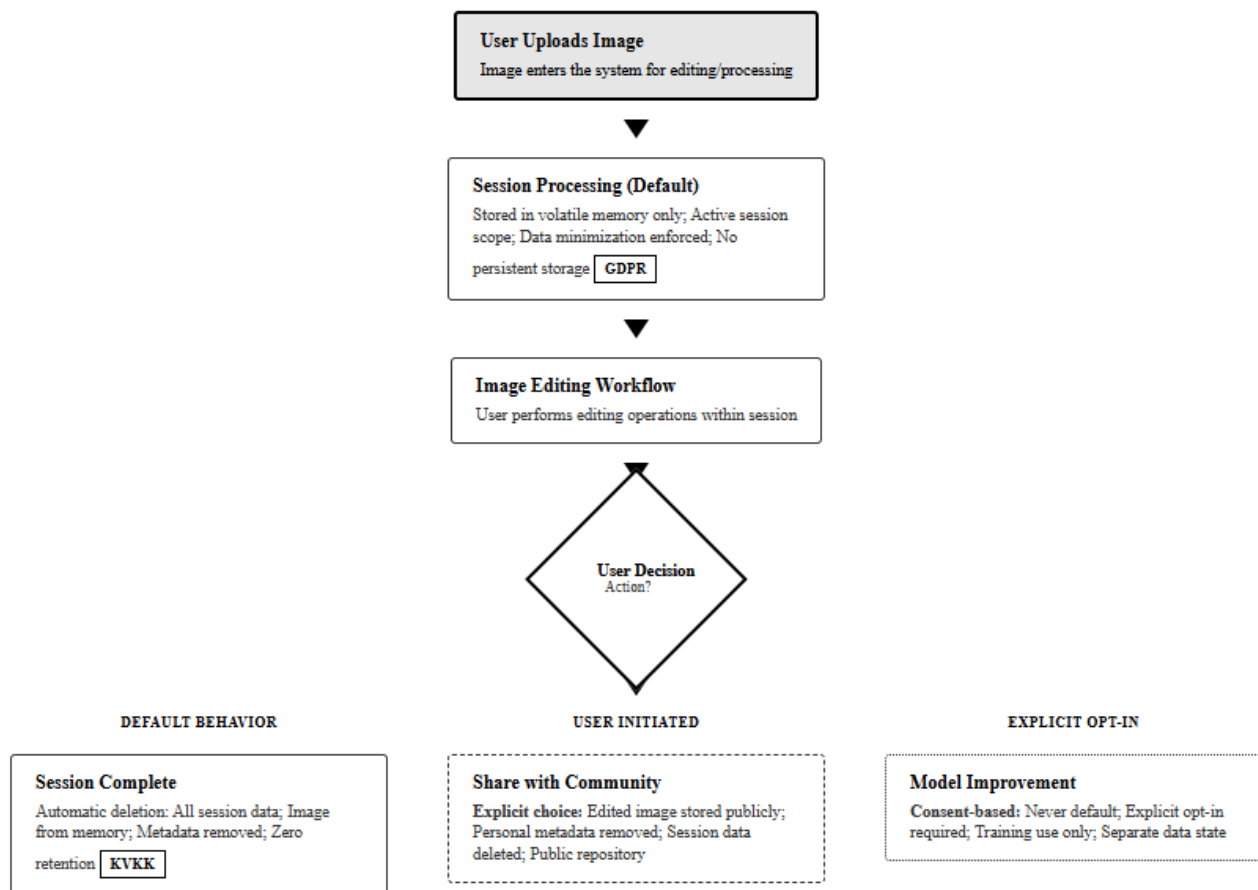
Invisio adopts a **privacy-first and purpose-driven data management approach** for handling user-uploaded images. By default, all image data is processed within the scope of an active session and stored only in volatile memory. Once the image editing workflow is completed,

session lifecycle management mechanisms automatically delete all session-related image data. This default behavior enforces data minimization, limits data exposure, and aligns with data protection principles defined by regulations such as GDPR and KVKK.

Persistent storage is applied only under explicit user control and for clearly defined purposes. If a user chooses to share an edited image with the community, the resulting image may be retained as public content in a dedicated repository after the removal of all personal and session-related metadata. Additionally, the system architecture supports optional reuse of user-provided images for model improvement strictly through explicit opt-in consent. Images are never retained or reused for training by default. Different data lifecycles—transient session data, community-shared public content, and explicitly authorized training data—are managed as separate data states, each governed by user intent and consent-based controls. This structured approach ensures transparent data governance, controlled persistence, and long-term regulatory compliance.

Figure 1. Invisio Data Process Flow Diagram

Privacy-First and Purpose-Driven Data Management Architecture



■ Entry Point □ Default Path (Privacy-First) ▤ User Choice (Optional) ▤ Explicit Opt-In Required

3.5. Access control and security

Invisio integrates security considerations directly into its architectural design rather than treating them as add-on features. All communication between client-side and server-side components is conducted over secure communication channels to protect data in transit. Incoming requests are subject to systematic input validation to prevent malformed, invalid, or potentially malicious data from entering the processing pipeline.

The system follows the **principle of least privilege** by limiting access to internal resources strictly to the components that require them for execution. Image data is exposed only within the scope of active processing and is not shared across system boundaries unnecessarily. This approach minimizes the attack surface and reduces the risk of unintended data leakage during AI-based processing.

Although user authentication and authorization mechanisms are not mandatory for the initial academic version of the system, the architecture is designed to support secure access control extensions in future iterations. By maintaining clear separation between user interaction, request handling, and processing services, the system can incorporate authentication, role-based access control, or token-based authorization mechanisms without requiring fundamental architectural changes.

3.6. Global software control

Global software control in Invisio is managed through a centralized coordination mechanism that governs request handling, task sequencing, and system-wide error propagation. All image processing requests are orchestrated through this control layer to ensure consistent execution behavior across concurrent users. The control flow is designed to operate asynchronously, allowing long-running AI inference tasks to be executed without blocking user interaction and enabling efficient utilization of backend resources.

To maintain system stability under varying load conditions, the control mechanism is designed to be **resource-aware**. When computational resources—particularly GPU capacity—are saturated, incoming requests may be queued, delayed, or gracefully rejected. This load-aware behavior prevents overload, ensures fair resource allocation, and avoids cascading failures during peak usage. The control flow also safely handles retried or duplicate requests, preventing inconsistent system states or unintended resource consumption caused by network interruptions or repeated user actions.

Key control events such as request initiation, failure detection, and task completion are designed to be traceable for monitoring and operational analysis purposes, without persisting personal image data. This approach improves system observability while preserving the system's privacy-oriented design.

3.7. Boundary conditions

The operation of the Invisio system is subject to a set of explicit boundary conditions that define its functional and operational limits. These constraints are considered during system design to ensure predictable behavior under non-ideal conditions.

The primary boundary conditions are as follows:

- **Network dependency:** A stable internet connection is required for communication between client-side and server-side components.
- **Input constraints:** Only predefined image formats and file size limits are accepted to ensure safe and efficient processing.
- **Resource dependency:** System performance depends on the availability of computational resources, particularly GPU capacity for AI inference.
- **Degraded operation:** Under constrained resource availability, the system may temporarily restrict AI-intensive features while preserving core functionality.
- **Intentional non-goals:** The system does not support real-time video processing, offline usage, or continuous background execution. These limitations represent deliberate design decisions rather than technical shortcomings.
- **Graceful failure handling:** When boundary conditions are violated, the system provides clear feedback and controlled error handling instead of failing unpredictably.

By explicitly defining these boundary conditions, Invisio ensures transparency regarding its operational limits and maintains reliability across a wide range of usage scenarios.

4. Subsystem services

Each subsystem in the Invisio system is responsible for providing a set of well-defined and limited services. This separation allows the system to remain modular and easier to maintain as new features and models are added over time. Subsystem services listed as **image submission, AI-based transformation execution, result visualization and delivery and system status reporting**.

The **image submission and validation service** is responsible for receiving image data from the user interface and performing basic checks before processing.

The **AI-based transformation execution service** handles the core image processing operations of the system. This service manages the invocation of AI models such as GAN-based object removal, image enhancement, or style transfer models. It is responsible for preparing input data, executing model inference, and producing the transformed output.

The **result visualization and delivery service** focuses on presenting the processed image back to the user in a clear and usable form. It prepares the output for display in the user interface and handles the delivery of results through response mechanisms.

The **system status reporting service** provides information about the current state of the system, such as processing progress, error conditions, or service availability. This information is used both for user feedback and for basic monitoring purposes, helping users understand the status of their requests and supporting troubleshooting during development and operation.

Service interfaces between subsystems are designed to be stable and clearly defined. This design choice ensures that internal changes within a subsystem, such as model upgrades or performance improvements, do not require modifications in other parts of the system. As a result, the overall architecture supports long-term maintainability, extensibility, and controlled system evolution.

5. Glossary

- **Artificial Intelligence (AI):** Computer systems that can perform tasks such as image analysis and automated editing.
- **Machine Learning (ML):** A method that allows systems to learn patterns from data without explicit programming.
- **Deep Learning:** A type of machine learning based on multi-layer neural networks, commonly used in image processing.
- **Computer Vision:** A field of AI focused on understanding and processing visual information from images.
- **Convolutional Neural Network (CNN):** A neural network architecture widely used for image feature extraction and segmentation.
- **Generative Adversarial Network (GAN):** A deep learning model consisting of a generator and a discriminator, used to create realistic images.
- **Inference:** The process of applying a trained AI model to input data to produce an output.
- **Image Segmentation:** The process of dividing an image into meaningful regions for analysis or editing.
- **Mask:** A region map that specifies which parts of an image will be modified during processing.

- **Inpainting:** An image editing technique used to fill removed or missing image regions.
- **Style Transfer:** An AI-based technique that applies the visual style of one image to another.
- **Human-in-the-Loop:** A system approach where limited user input is used to improve AI-generated results.
- **Hybrid Editing Workflow:** A workflow that combines automated AI processing with optional user correction steps.
- **Session-Based Processing:** A processing model where user data exists only during an active session.
- **Stateless System:** A system that does not store user-specific information between requests.
- **Volatile Memory:** Temporary memory used only during processing and cleared afterward.
- **Privacy-by-Design:** A design principle where privacy and data protection are considered from the beginning.
- **Application Programming Interface (API):** A communication interface that allows software components to interact with each other.
- **User Interface (UI):** The part of the system through which users interact with the application.
- **Asynchronous Processing:** A processing approach where tasks run without blocking the user interface.
- **Subsystem:** A functional component of the system responsible for a specific task.
- **Modular Architecture:** A design approach where system components are loosely coupled and independently manageable.
- **Transport Layer Security (TLS):** A protocol used to secure data communication between client and server.
- **GDPR:** A European regulation that defines rules for personal data protection.
- **KVKK:** Turkey's law for the protection of personal data.

6. References

- Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., et al. (2014). *Generative adversarial nets*. NeurIPS.
- Gatys, L. A., Ecker, A. S., & Bethge, M. (2015). *A neural algorithm of artistic style*. arXiv preprint.
- Jing, Y., Yang, Y., Feng, Z., Ye, J., Yu, Y., & Song, M. (2017). *Neural style transfer: A review*. IEEE Trans. on Visualization and Computer Graphics.
- Xu, Y. (2025). *Style transfer review: Traditional machine learning to deep learning*. Information, 16(2).
- Zhang, S., et al. (2025). *Applying deep learning for style transfer in digital art*. Scientific Reports.

- Diamant, N., Zadok, D., Baskin, C., Schwartz, E., & Bronstein, A. M. (2019). *Beholder-GAN: Generation and beautification of facial images with conditioning on their beauty level*. In *Proc. IEEE ICIP*.
- Gan, J., et al. (2024). *Applied research on face image beautification based on FIBGAN*. Electronics.
- Peng, T. (2024). *ISFB-GAN: Interpretable semantic face beautification*. Expert Systems with Applications.
- Ronneberger, O., Fischer, P., & Brox, T. (2015). *U-Net: Convolutional networks for biomedical image segmentation*. MICCAI.
- Ling, H., Kreis, K., Li, D., Kim, S. W., Torralba, A., & Fidler, S. (2021). *EditGAN: High-precision semantic image editing*. arXiv preprint.
- Anwar, S., Tahir, M., Li, C., Mian, A., Khan, F. S., & Muzaffar, A. W. (2020). *Image colorization: A survey and dataset*. arXiv preprint.
- [Author(s)] (2024). *[Survey on image colorization]*. Electronics.
- Gui, J., Sun, Z., Wen, Y., Tao, D., & Ye, J. (2020). *A review on Generative Adversarial Networks: Algorithms, theory, and applications*.
- Shamsolmoali, P., Zareapoor, M., Granger, E., Zhou, H., Wang, R., Celebi, M. E., & Yang, J. (2020). *Image synthesis with adversarial networks: A comprehensive survey and case studies*. arXiv preprint.
- Wu, X., Xu, K., & Hall, P. M. (2017). *A survey of image synthesis and editing with generative adversarial networks*. Tsinghua Sci. & Tech.
- Zhou, X. (2025). *Survey paper bridging the metrics gap in image style transfer*. [Journal].
- Heusel, M., Ramsauer, H., Unterthiner, T., Nessler, B., & Hochreiter, S. (2017). *GANs trained by a two time-scale update rule converge to a local Nash equilibrium*. NeurIPS.
- [Author(s)] (2025). *A review on generative AI for text-to-image and image-to-image generation*. arXiv preprint.